Susanne L, Alexander L

Chain of Responsibility

1. Name

Name	Chain of Responsibility
Kategorie	Verhaltensmuster (Behavioral Pattern)
Kurzfassung	Eine Anfrage wird entlang einer Kette von Handlern weitergereicht, bis einer sie bearbeitet.

2. Beschreibung

Das Chain-of-Responsibility-Pattern dient dazu, eine **lose Kopplung** zwischen dem Sender einer Anfrage und ihren Empfängern herzustellen. Mehrere Objekte (die *Handler*) werden zu einer Kette verbunden. Der Client schickt seine Anfrage an das erste Glied der Kette. Jedes Glied entscheidet selbst, ob es die Anfrage bearbeitet oder an seinen Nachfolger weitergibt.

Der Client weiß dabei nicht, welches Glied die Anfrage am Ende verarbeitet. Dadurch lassen sich Handler hinzufügen, entfernen oder umordnen, ohne den Client zu ändern. Ein bekanntes Beispiel ist der `try-catch`-Block mit mehreren `catch`-Zweigen: Jeder Zweig behandelt eine bestimmte Exception, alles andere wird nach außen weitergereicht.

3. Beispielproblem: Ticketsystem im Fußballstadion

Für ein Heimspiel werden Eintrittskarten aus verschiedenen Kontingenten vergeben: Dauerkarteninhaber haben ein Vorkaufsrecht, Vereinsmitglieder haben ein eigenes Kontingent, der Rest geht in den freien Verkauf. Ein Kunde möchte zum Beispiel zwei Tickets kaufen. Das System muss entscheiden, aus welchem Kontingent die Anfrage bedient wird, und kann die Anfrage auf eine Warteliste setzen, wenn nichts mehr verfügbar ist.

Die naheliegende Lösung ist ein zentraler Ticketservice mit einer langen if/else-Struktur (Dauerkarte? Mitglied? freier Verkauf? sonst Warteliste). Das führt zu mehreren Problemen:

- Die gesamte Logik steckt in einer Methode; die Komplexität wächst mit jeder neuen Regel.
- Der Code ist **eng gekoppelt**: Kommt ein Sponsoren- oder Gästefan-Kontingent hinzu, muss bestehender Code geändert werden.
- Reihenfolge und Zuständigkeiten sind fest verdrahtet. Bei einem Spiel ohne freien Verkauf müsste die Methode wieder angepasst werden.
- Die einzelnen Regeln lassen sich schwer wiederverwenden und getrennt testen.

4. Lösungsbeschreibung

Je nach Kunde (Stammkunde, Fan, Dauerkarte etc.) müssen andere Funktionen verwendet werden. Des Weiteren wird beim Zahlungsverkauf eine Reihe von Prozessen durchlaufen, die mehr oder weniger standardisiert sind.

5. Vorteile

- **Lose Kopplung:** Der TicketShop kennt nur den ersten Handler, nicht die einzelnen Kontingente.

- **Flexibilität:** Ein neues Kontingent (z. B. Sponsoren oder Gästefans) ist eine neue Klasse und ein Eintrag beim Kettenaufbau; der Client bleibt unverändert.
- **Single Responsibility:** Jeder Handler kennt genau eine Regel und ein Kontingent.
- **Konfigurierbarkeit:** Die Kette lässt sich pro Spiel anders zusammenstellen, etwa ohne freien Verkauf bei einem Spiel.
- **Testbarkeit/Erweiterbarkeit:** Handler können einzeln getestet und in anderen Ketten wiederverwendet werden.

6. Struktur (UML-Diagramm)

Allgemeine Struktur

Rolle	Aufgabe
Handler	Gemeinsame abstrakte Klasse bzw. Interface: Bearbeitungsmethode und Referenz auf den Nachfolger (next).
ConcreteHandler	Prüft, ob er zuständig ist; bearbeitet die Anfrage oder delegiert an next.
Client	Baut die Kette auf und sendet die Anfrage an das erste Glied.

